

Baboons Ears

Complete User Guide

Offline field-training manual for the baboon ear identification PWA
Generated on 2026-06-19

This guide explains what is inside the app, how the study modes work, how the mark codes are displayed, how to install the app on phone or computer, and how to keep it available without internet.

1. What This App Is

Baboons Ears is a lightweight progressive web app (PWA) designed for baboon ear-mark training. It lets a user browse known individuals, study them as flashcards, and keep simple training statistics. The app is built to keep working offline after the first successful online load.

- Browse all individuals as visual cards with ear photos, troop, age/sex, and mark notation.
- Study in Normal mode: ears to name.
- Study in Hard mode: mark notation to name.
- Answer either with a four-choice quiz or by typing the baboon name.
- Filter study sessions by troop, age group, and session size.
- Track overall accuracy and review the most recent sessions.
- Run offline once the app shell, dataset, and images have been cached on the device.

2. What Is Included in the Current Dataset

The current app dataset comes from the exported training material and a supplementary PDF-based image extraction pass. The figures below reflect the files currently present in the project.

Dataset item	Value
Deck name	Baboons Ears
Total cards	125
Troop J cards	55
Troop L cards	70
Adult cards	77
Juvenile cards	40
Subadult cards	8
Cards with no image	0
Cards with one visible ear image only	1

Age/sex categories currently present in the app are Adult Female, Adult Male, Juvenile Female, Juvenile Male, and Subadult Male.

Age / sex	Cards
Adult Female	58
Adult Male	19
Juvenile Female	22
Juvenile Male	18
Subadult Male	8

Known partial record: Sarcomainf24 currently has one visible ear image in the project because only one clear ear image was available from the source material used for supplementation.

3. Main Sections of the App

- Browse: a searchable visual catalogue of all baboons in the dataset.
- Flashcards: the active training section where study sessions happen.
- Field Guide: a short in-app reminder about offline installation and use.
- Progress: a summary of total accuracy plus the most recent training sessions.

4. Browse Mode

Browse mode is the reference view. It is useful when someone wants to inspect individuals freely before starting a timed or structured study session.

- Search by name, troop, age/sex, or mark text.
- Filter by troop.
- Filter by age/sex.
- See each card with one or two ear images, the baboon name, troop, age/sex, and mark label.
- Clear all browse filters with one button.

5. Flashcard Study Modes

The study section uses a flashcard logic rather than a classic linear quiz. Each card presents one prompt, reveals the correct answer, then lets the user move to the next card.

- Normal (ears to name): the front of the card shows the ear photos and the user must identify the baboon.
- Hard (marks to name): the front of the card shows only the mark notation, for example 'R: TB • L: MB', and the user must identify the baboon from the notation alone.
- Quiz (4 choices): the app proposes four names and highlights green for the correct answer and red for a wrong selected answer.
- Write the name: the user types the answer manually and the app checks it against the stored baboon name.

6. Study Filters and Session Controls

- Troop filter: All troops, J only, or L only.
- Age-group filter: All ages, Adult only, Juvenile only, or Subadult only.
- Session size: 10 cards, 20 cards, 50 cards, or All cards.
- Start session: builds a new queue from the currently allowed study pool.

A session stops when the chosen target size has been reached. If the user selects 10 cards, the app records a 10-card session rather than continuing indefinitely.

7. Intelligent Card Rotation

The app uses a simple weighted system so difficult baboons return more often. This is not random repetition only: it is designed to bring back individuals that were missed more often or that do not yet have a good success streak.

- Each card stores how many times it has been seen, answered correctly, answered wrongly, and how long the current success streak is.
- Wrong answers increase a card's future weight strongly.
- Short streaks also increase the chance of coming back soon.
- The result is a practical field-training loop where weak points are reviewed more often than easy individuals.

8. Progress Tracking

Progress is stored locally on the device through browser local storage. It stays on that same device unless the browser data is cleared.

- Overall accuracy percentage across completed sessions.
- Total number of correct answers out of total reviewed cards.
- A visual progress bar.
- A recent-session history showing mode, answer style, score, session size, troop filter, and age filter.
- Per-card memory used by the repetition logic.

9. Understanding the Ear-Mark Notation

The app preserves the original mark notation from the source deck. It does not invent or reinterpret the ear-mark strings. This is important because the app is meant to train the user on the exact codes already used in the field material.

- R means right ear.
- L means left ear.
- A label such as 'R: TB • L: MB' means the right-ear code is TB and the left-ear code is MB.
- A value of '0' means no specific code was recorded for that ear in the source material.
- Combined strings such as TB, TMB, TTMB, sT, or sTB are displayed exactly as they appear in the exported dataset.
- In Hard mode, these notation strings are the prompt the user studies from.

Important caution: the app can document and display the code vocabulary, but the exact biological or field-definition expansion of each letter string should be confirmed against the project's official ear-notch legend if you need a formal field protocol. In other words, the app is faithful to the source codes, but it is not itself the authoritative decoding key for the letters.

10. Mark Code Vocabulary Seen in the Current Dataset

The following ear-code values are currently present in the dataset. They are listed exactly as stored in the source-derived JSON.

Code	How to read it in the app	Interpretation status
0	Shown literally as '0' on the relevant ear	No recorded code for that ear
B	Shown literally as 'B' on the relevant ear	Source code preserved; confirm official legend for exact meaning
M	Shown literally as 'M' on the relevant ear	Source code preserved; confirm official legend for exact meaning
MB	Shown literally as 'MB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
MoB	Shown literally as 'MoB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
MsB	Shown literally as 'MsB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
T	Shown literally as 'T' on the relevant ear	Source code preserved; confirm official legend for exact meaning
T.s.M.s	Shown literally as 'T.s.M.s' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TB	Shown literally as 'TB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TBB	Shown literally as 'TBB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TM	Shown literally as 'TM' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TMB	Shown literally as 'TMB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TMsB	Shown literally as 'TMsB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TT	Shown literally as 'TT' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TTMB	Shown literally as 'TTMB' on the relevant ear	Source code preserved; confirm official legend for exact meaning

TTMMB	Shown literally as 'TTMMB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
TsB	Shown literally as 'TsB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
sB	Shown literally as 'sB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
sM	Shown literally as 'sM' on the relevant ear	Source code preserved; confirm official legend for exact meaning
sT	Shown literally as 'sT' on the relevant ear	Source code preserved; confirm official legend for exact meaning
sTB	Shown literally as 'sTB' on the relevant ear	Source code preserved; confirm official legend for exact meaning
sTM	Shown literally as 'sTM' on the relevant ear	Source code preserved; confirm official legend for exact meaning
sTMB	Shown literally as 'sTMB' on the relevant ear	Source code preserved; confirm official legend for exact meaning

11. Example Study Prompts

- Normal example: two ear photos are shown; the user must identify the baboon's name.
- Hard example: 'R: TBB • L: T' appears; the user must recall the correct name from the notation.
- Quiz example: four names are offered; selecting the correct one turns the right choice green.
- Typed-answer example: the user enters a name manually and the app compares it case-insensitively.

12. How to Install on iPhone

1. Deploy the project to a web host such as GitHub Pages.
2. Open the site in Safari while the phone is online.
3. Wait for the first full load so the app shell, JSON, and images can cache.
4. Tap Share, then choose Add to Home Screen.
5. Open the app once from the new icon while still online.
6. Test it with Airplane Mode enabled before relying on it in the field.

Important iPhone note: iOS can be more aggressive than Android about clearing cached website data if storage is low or if the app is unused for a long time. Always test the installed app on the real device shortly before field use.

13. How to Install on Android

7. Deploy the project to a web host such as GitHub Pages.
8. Open the site in Chrome or another Chromium-based browser while online.
9. Wait for the first full load.
10. Choose Add to Home Screen or Install App when offered by the browser.
11. Launch it once from the installed icon before going offline.
12. Test it with Wi-Fi and mobile data both turned off.

Android is generally more permissive and reliable for offline PWAs, especially when the image cache has been warmed successfully on first use.

14. How to Use It on a Computer

- Simplest option: use the deployed website in a desktop browser.
- For Chrome or Edge, you can usually install it as a desktop app from the browser menu.

- If you are working locally, open it through a local web server rather than `file://` because service workers need an HTTP context.
- A simple local server can be started from the project folder with `'python3 -m http.server 8000'`, then opened at `http://localhost:8000`.

On desktop, offline behavior still depends on first loading the app successfully so the service worker and cached assets are available.

15. How Offline Mode Works

This project is built as a progressive web app. A service worker caches the app shell files, the JSON dataset, and the ear photos. After that first successful caching step, the app can reopen without network access.

- Core cached files include the HTML, CSS, JavaScript, manifest, service worker, dataset JSON, and icon files.
- Image warm-up logic also adds ear images to the browser cache.
- The connection indicator changes between an online/cached state and an offline state.
- If the device has never finished the first online load, offline use may be incomplete.

16. How to Update the Website After Changes

13. Edit the project files locally.
14. Commit and push the changes to the GitHub repository.
15. Wait for GitHub Pages to redeploy.
16. Refresh the live website on the device.
17. If an old version seems stuck, close the installed app fully and reopen it.
18. If needed, remove the home-screen app and install it again to force a clean refresh.

Because PWAs use cached assets, a browser can occasionally keep an older version longer than expected. Reopening after the service-worker cache version changes usually resolves that.

17. Known Limits and Good Practice

- The app stores training progress locally on one device; it is not a cloud-synced score system.
- If browser data is cleared, the local progress history is lost.
- The exact semantic decoding of every mark string should be validated against the team's official notch legend.
- One record currently has only one visible ear image in the project files.
- Before field deployment, always test on the real phone in offline mode.

18. Quick Glossary

Term	Meaning
Browse	The visual catalogue of all cards.
Flashcard	A study card with a front prompt and a revealed answer.
Normal mode	Ears to name.
Hard mode	Mark notation to name.
Quiz mode	Four answer choices.
Write the name	Free-text answer entry.
Troop	Group label currently shown as J or L.
Age / Sex	Detailed category such as Adult Female or Juvenile Male.
Age group	Broader filter: adult, juvenile, or subadult.
Session size	How many cards are included in one study run.
Offline or cached	The app has enough local files stored to run without network access.

19. File Structure in the Project

- index.html: main page structure.
- style.css: layout and visual styling.
- app.js: data loading, browse filters, flashcard logic, progress tracking, and cache warm-up.
- manifest.json: install metadata for the PWA.
- service-worker.js: offline caching logic.
- data/individuals.json: generated dataset with names, groups, marks, and image paths.
- images/: ear-photo assets used by the cards.
- scripts/: helper scripts for rebuilding data and generating project outputs.

20. Final Summary

In practice, this app is a focused offline study tool for baboon identification. It combines ear photographs, source mark notation, flashcard modes, device-local progress, and PWA offline delivery. Once deployed and installed correctly, it provides a cleaner field-training workflow than a generic quiz tool because it respects the troop filters, age-group filters, ear-mark prompts, and repeated review of difficult individuals.